

Simulated Fouls | Project 01

This file simulates NBA game data that will reflect our project' data generating process: - Total fouls called in a game, driven by the officiating crew plus game context (team foul/pace tendencies, home/away, playoff flag). - Foul counts are non-negative, whole numbers. - We generate from a Poisson distribution whose mean depends on crew and context - Will mirror Poisson regression model planned to fit to real data.

Output: simulated_games.csv (one row per game).

Imports

```
import numpy as np
import pandas as pd
# from numpy.char import index
# from openpyxl.worksheet.table import _TableColumnTotalsRowFunction
```

Reproducible Results

```
rng = np.random.default_rng(42)
```

Game Settings (Fake)

```
nGames = 1000          ## how many games to simulate
nCrews = 15             ## number of distinct officiating crews
nTeams = 30             ## NBA has 30 teams
playoffShare = 0.15     ## fraction of games that are playoff games
```

Crew's Foul Tendencies

- Some crews will call more (tight), some will call fewer (loose)
- We're pretending referees have real, consistent differences

```
crewEffect = rng.normal(loc=0.0, scale=0.12, size=nCrews)
```

Teams Foul Tendencies (How many they tend to commit)

- Some teams will commit more fouls than others

```
teamEffect = rng.normal(loc=0.0, scale=0.08, size=nTeams)
```

Baseline

```
baseline = 42
```

Main Logic

- Start at league average of 42 fouls, nudge up/down based on who's officiating, who's playing, and if playoffs.
- Result: expected number of fouls for specific game.

```
rows = []
for game in range(nGames):
    crew = rng.integers(0, nCrews)
    homeTeam = rng.integers(0, nTeams)
    awayTeam = rng.integers(0, nTeams)
    while awayTeam == homeTeam:
        awayTeam = rng.integers(0, nTeams)

    isPlayoff = rng.random() < playoffShare

    ## Building Expected Number of Fouls on Log Scale
    logMean = np.log(baseline)
    logMean += crewEffect[crew]                ## crew's fingerprint
    logMean += teamEffect[homeTeam]            ## home team tendency
    logMean += teamEffect[awayTeam]            ## away team tendency
    logMean += -0.06 if isPlayoff else 0.0     ## refs call fewer fouls
    in playoffs
    logMean += -0.02                            ## small home-court effect
    (fewer fouls on home)

    expectedFouls = np.exp(logMean)

    ## Adding real world randomness - real games aren't exactly average
    ## Adds randomness, so our data will look messy and realistic
    totalFouls = rng.poisson(expectedFouls)    ## draw actual foul count from
    a Poisson with mean

    rows.append({
        "CrewID": crew,
        "HomeTeam": homeTeam,
```

```

        "AwayTeam": awayTeam,
        "HomeAway": "home",                ## row is written from home
game's perspective
        "isPlayoff": int(isPlayoff),
        "TotalFouls": totalFouls,          ## countinuous target
    })

df = pd.DataFrame(rows)

```

Printing to Screen

```

print('First 10 Simulated Games:')
print(df.head(10).to_string(index=False))
print()
print('Summary of Total Fouls:')
print(df["TotalFouls"].describe().round(2).to_string())
print()
print(f'Saved {len(df)} games to simulated_games.csv')

df.to_csv('simulated_games.csv', index=False)

```

First 10 Simulated Games:

CrewID	HomeTeam	AwayTeam	HomeAway	isPlayoff	TotalFouls
13	8	7	home	1	42
10	23	19	home	0	51
7	17	1	home	1	40
2	25	16	home	0	49
1	16	23	home	1	30
4	8	12	home	0	28
12	8	27	home	0	40
4	29	19	home	0	41
5	0	3	home	0	35
1	27	15	home	0	45

Summary of Total Fouls:

count	1000.00
mean	41.52
std	8.41
min	20.00
25%	35.00
50%	41.00
75%	47.00

max 70.00

Saved 1000 games to simulated_games.csv